Security Protocols

Iulian Aciobanitei, Phd

Bucharest

2024

Schedule

1. Vulnerable Protocols
   - Traffic Analyzers
   - Insecure Protocols
2. Security Protocols
   - SSL
   - IPSEC
   - SMIME
   - PGP
3. Authorization Protocols
   - OAuth 2.0
   - JWT

Traffic Analyzers

- Wireshark
- tcpdump
- windump
- awk
- Netflow

Insecure Protocols

- FTP
- SMTP
- SMIME
- PGP

Overview

- Recap
- Advantages
- Usage
- Mutual TLS

Demo - Windows

- Generate server certificate
- Generate client certificate
- Virtual directory
- Activate TLS
- Mutual TLS
- Errors

Workshop

- Generate server certificate
- Generate client certificate
- Virtual directory
- Activate TLS
- Mutual TLS

Strongswan Overview

- URLs
  - Examples:
  - Strongswan.conf
- Configuration files
- Key exchange options

Demo

- Host-to-host
- AH, ESP
- Encapsulate multiple protocols

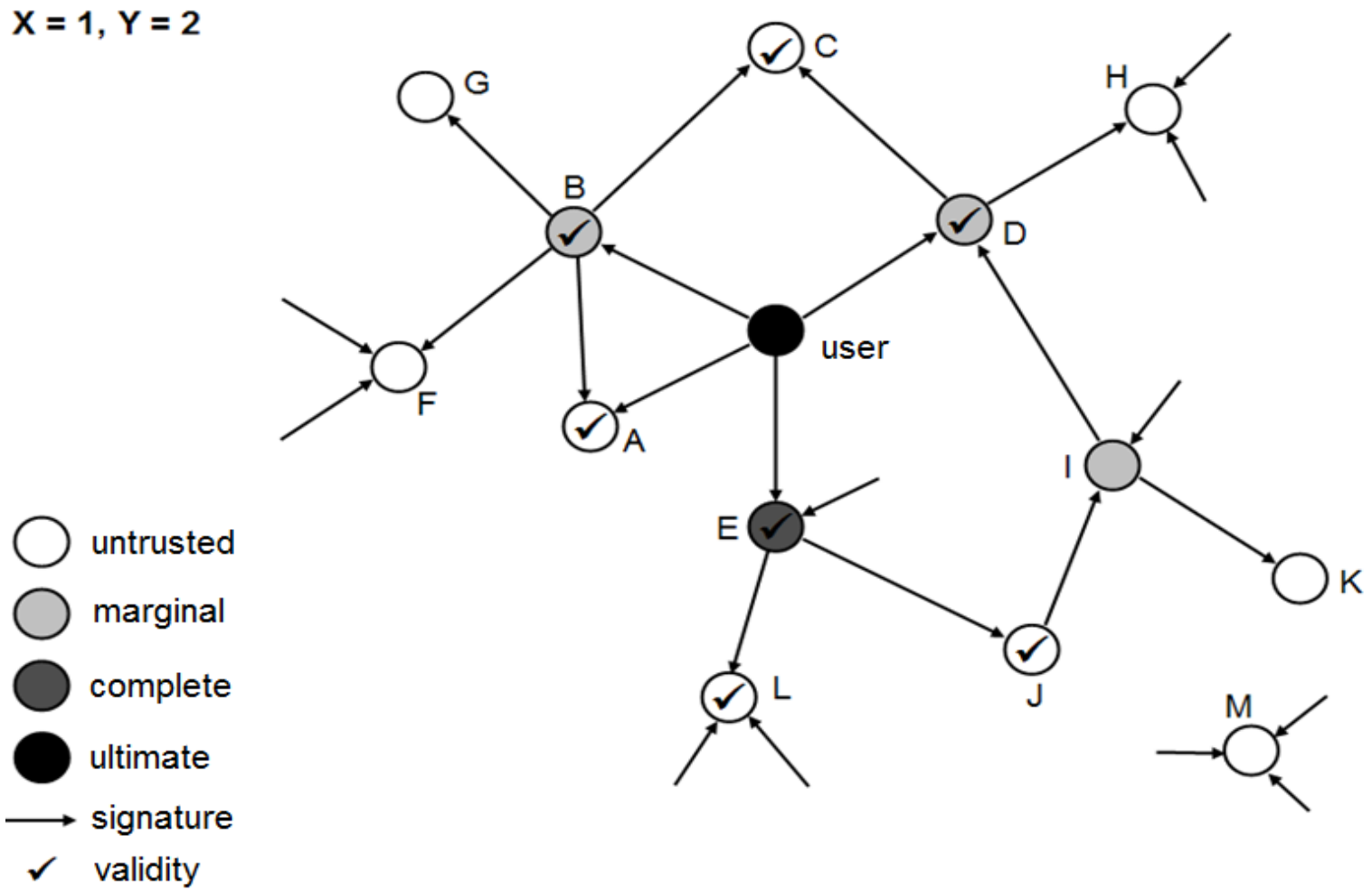
Workshop

- Choose the pairs
- Connect to the VMs
- Create the tunnels

Demo

- Email signing
  - All good
  - Tampered email
- Email encryption
- Outlook's Certificate Management

Web of trust


X = 1, Y = 2
G
C
H
B
D
user
F
A
I
E
K
L
J
M
untrusted
marginal
complete
ultimate
signature
validity

Demo

- PGP App Installation (https://www.gpg4win.org/)
- Key & cert generation
- Sign
- Encrypt
- Trust Transfer

Workshop

- Installation PGP App
- Generate Key & cert
- Sign
- Encrypt
- Exchange certs to create the Web Of Trust
- Validate a Signature

What is it?

- OAuth 2.0 is an open authorization framework
- Enables third-party applications to access resources securely
- Common use cases: APIs, mobile apps, single sign-on (SSO)
- Authorization vs authentication

Standards Defining OAuth 2.0

- **RFC 6749:** The OAuth 2.0 Authorization Framework
- **RFC 6750:** Bearer Token Usage
- **RFC 6819:** Threat Model and Security Considerations

OAuth 2.0 - Main Actors

- **Resource owner** - the entity that grants access to the protected resource
- **Resource Server** - server to hold the resource
- **Client** - An application that requests access to the resource
- **Authorization server** - Server to check credentials provided by the claimed Resource Owner and provide an access token to allow the access

OAuth 2.0 - Flows

- **Authorization Code Grant:** Used by secure server-side apps
  - Source code is not exposed publicly
- **Implicit Grant:** Deprecated; used for single-page apps.
- **Client Credentials Grant:** Machine-to-machine
  - use clientID and secret for access to a scope.
- **Resource Owner Password Grant:** Legacy apps
  - User inserts in client app credentials(discouraged)
- **Device Authorization Grant:** Devices with limited input.
  - User go to a link on a PC/mobile device.
- **PKCE** - extension on OAuth 2.0
  - Used for public clients that cannot reliably store client secrets.

OAuth 2.0 - Main benefits

- Scalable for large-scale systems.
- Enables Single Sign-On (SSO).
- Granular access control with scopes.
- Decouples authentication from authorization.
- Cross-platform usability.
- **Clients do not have access to credentials**
  - Use-case: Remote digital Signatures

Known Security Breaches

- Token leakage through URL in Implicit Flow.
- Misconfigured redirect URIs causing open redirects.
- Replay attacks in non-secure channels.
- Cross-Site Request Forgery (CSRF).

OAuth 2.0 - Security Considerations

- Use PKCE with Authorization Code Grant.
- Always secure communication with TLS/HTTPS.
- Validate redirect URIs to prevent open redirects.
- Rotate client secrets and tokens regularly.
- Mitigate CSRF using state parameters.

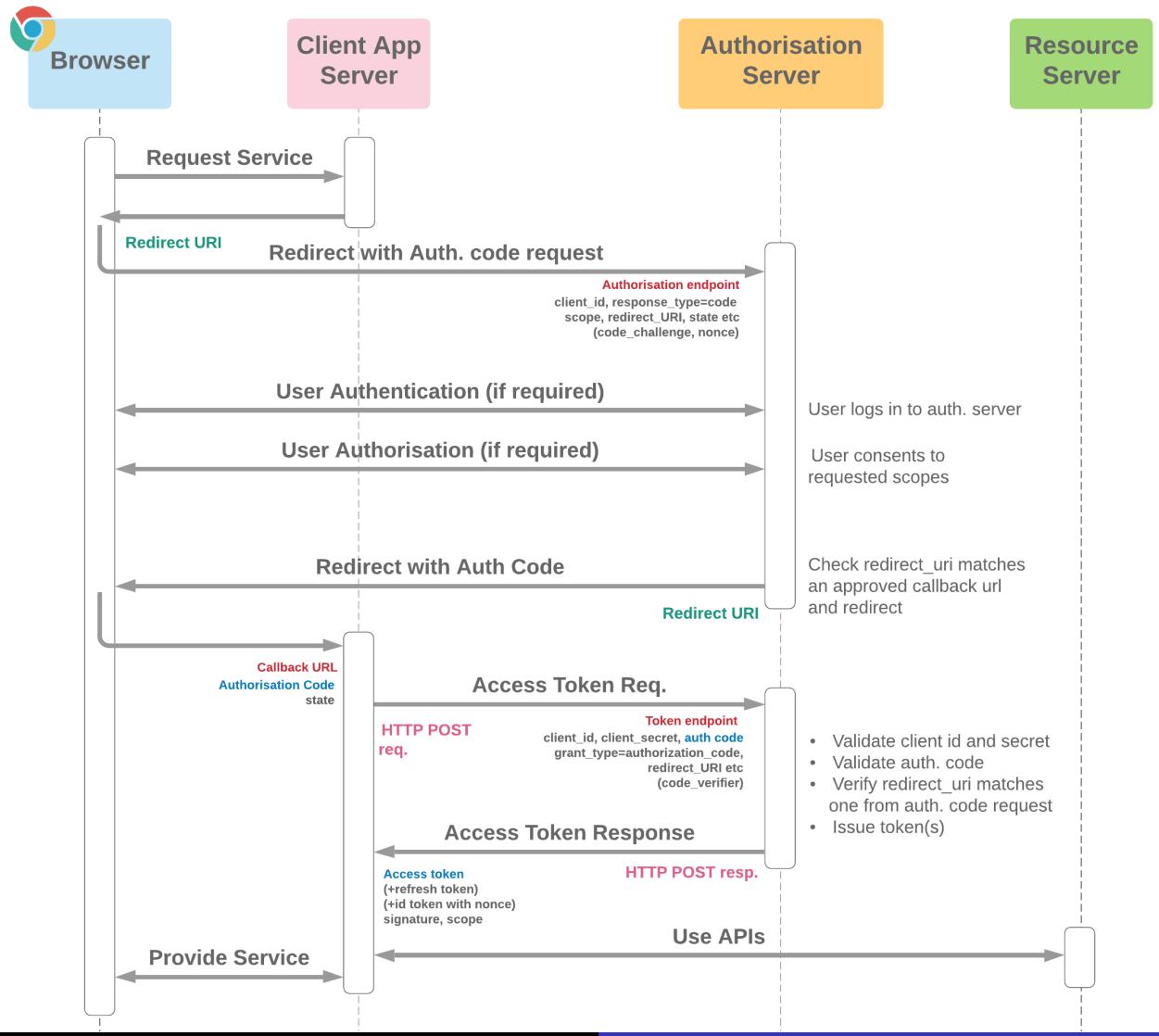
OAuth 2.0 and Single Sign-On (SSO)

- OAuth 2.0 underpins modern SSO.
- Users log in via a central Identity Provider.
- Simplifies credential management for users & clients.
- Reduces authentication complexity for developers.

POC Implementation Overview

- Use Google as the Identity Provider.
- Develop a simple frontend + backend app for OAuth 2.0 login.
- Demonstrate the Authorization Code Grant.

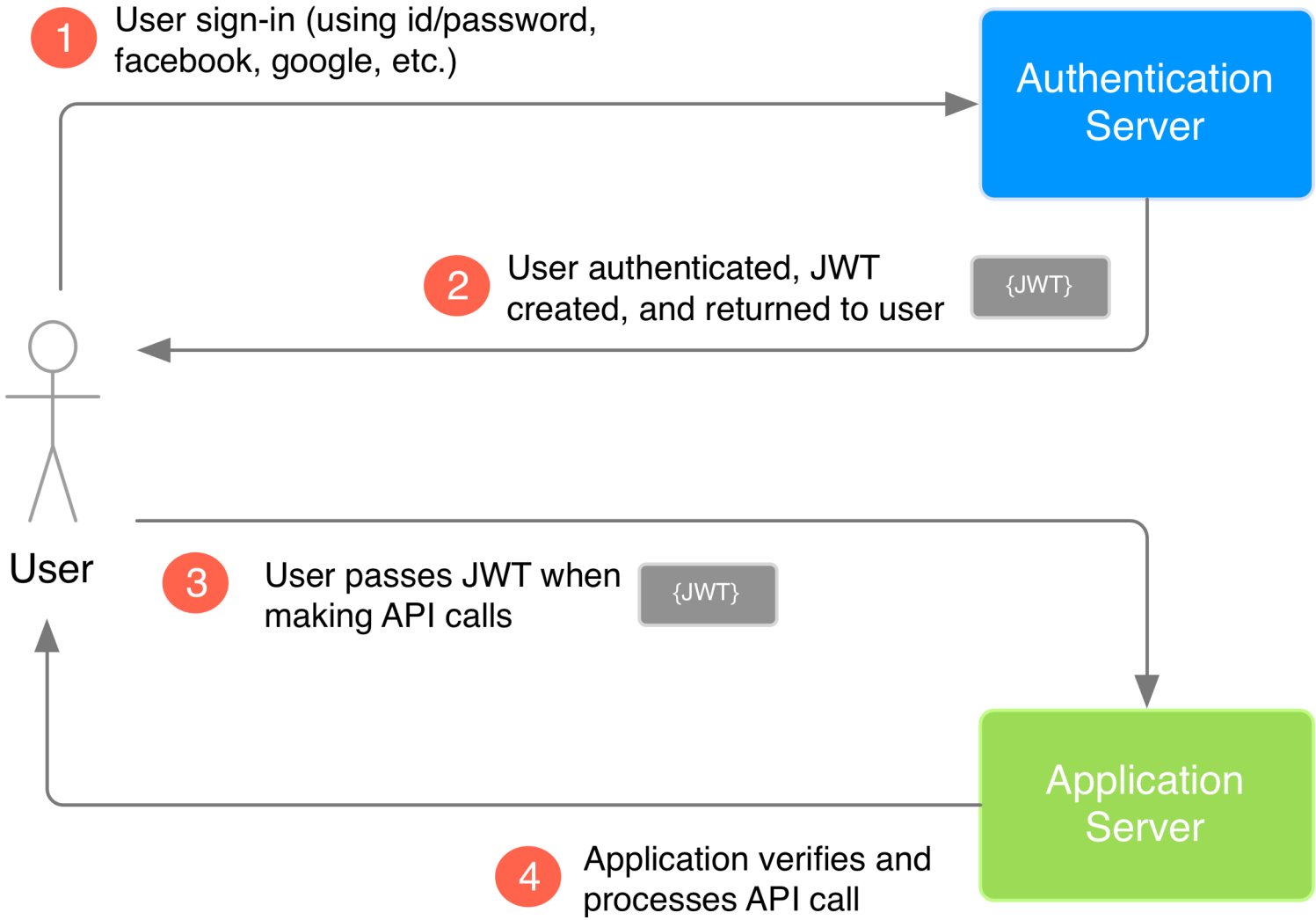
OAuth 2.0 - Authorization code flow



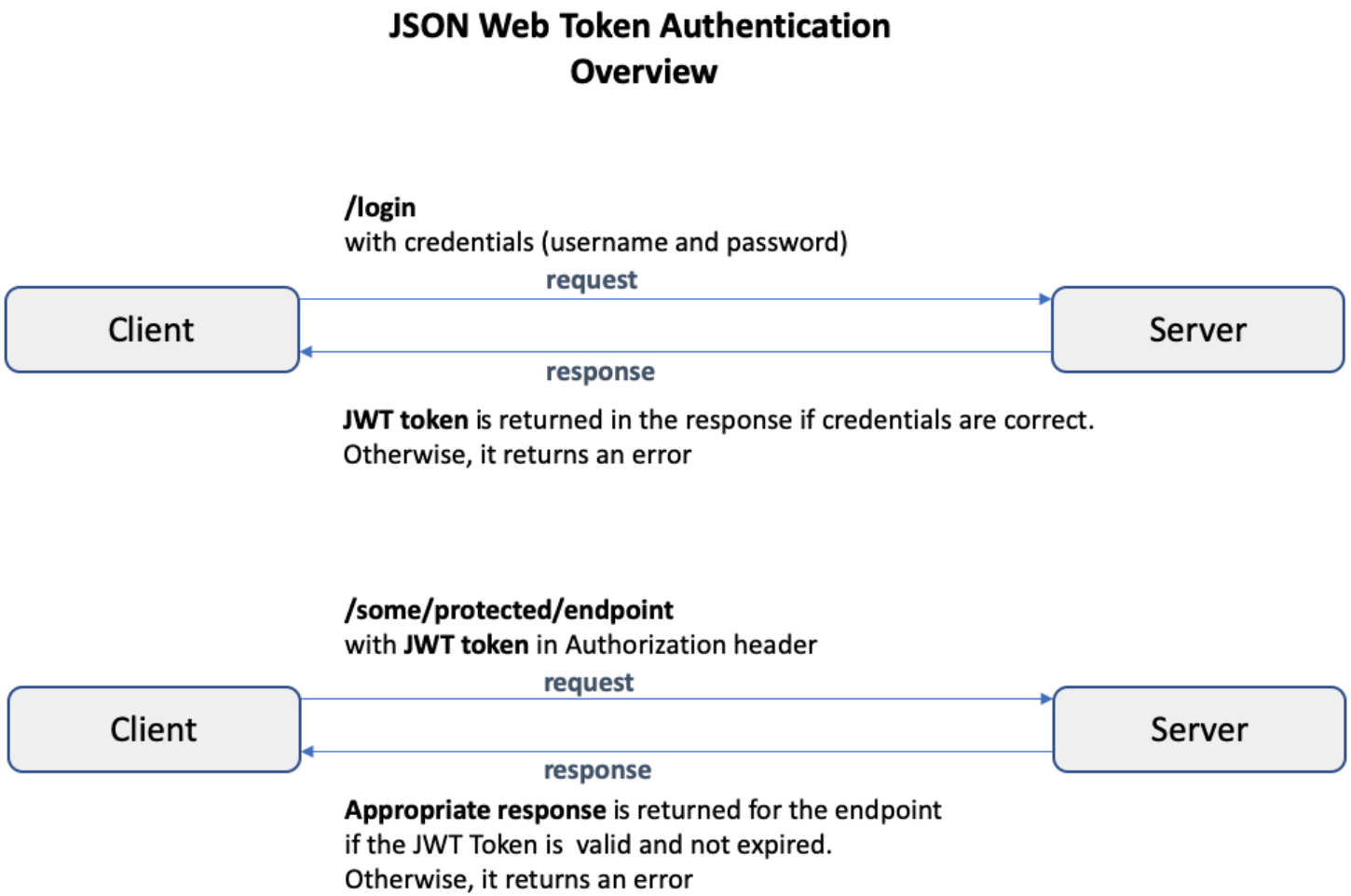
Demo

- Demo with Overleaf
- Discussion on AdobeSign/DocuSign

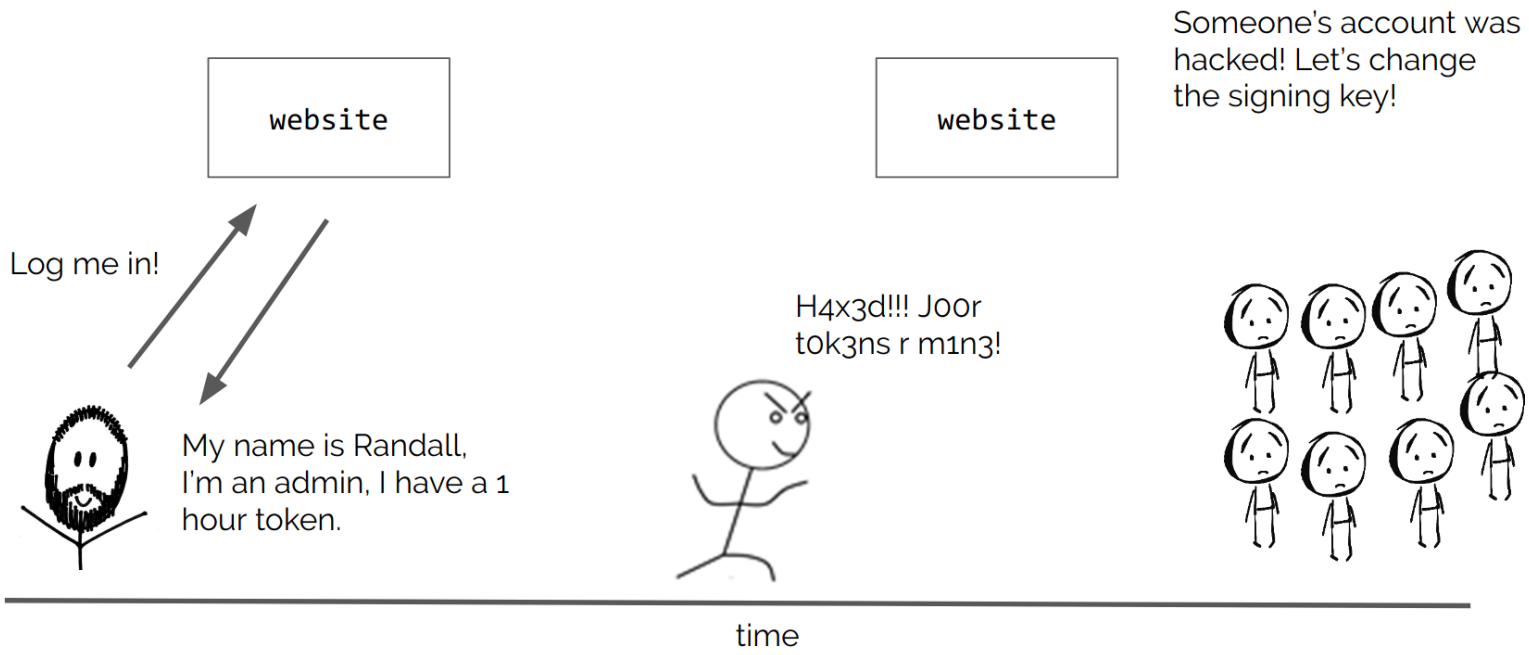
Overview


1
User sign-in (using id/password, facebook, google, etc.)
Authentication Server
2
User authenticated, JWT created, and returned to user
{JWT}
User
3
User passes JWT when making API calls
{JWT}
Application Server
4
Application verifies and processes API call

Resource Protection



Key renewal


website
website
Someone's account was hacked! Let's change the signing key!
Log me in!
My name is Randall, I'm an admin, I have a 1 hour token.
H4x3d!!! J00r t0k3ns r m1n3!
time

Introduction & Demo

- jwt.io
- Demo
- BurpSuite

Workshop

- Start Alice VM
- Start webgoat vm (run ./start_webgoat.sh from Desktop)
- Access http://localhost:8080/WebGoat/login and create an account
- Go to lesson (A2) Broken Authentication -> JWT Tokens